GraphOne / FrontierAtlas — Architecture Document

Scope: 3-day take-home trial. This run collected real records from 5 live, first-party APIs/feeds (YC/Algolia, arXiv, GitHub, 5 RSS news feeds, RemoteOK + 4 job-board APIs/feeds) at the trial's reduced volume. This document argues how the same architecture scales to the brief's full 500k+ target, and documents the real numbers this run actually produced.

Entity	Trial target (TRD.md)	This run
Startups	150–300	200
Products	150–300	200
Research Papers	200–400	300 (113 GitHub-matched, 187 honest `null`)
Jobs (24h fresh)	uncapped	226 (out of 413 raw across RemoteOK + 4 boards)
News (24h fresh)	uncapped	3 (out of 66 raw across all 5 feeds)

The News count is genuinely small, not under-filtering: checked at write time, only TechCrunch had anything under 24h old (newest 15.3h); The Verge's newest was 26.9h (just outside the window), Ars Technica and MIT Tech Review were 40+h stale, and VentureBeat intermittently returns zero entries under its WAF. A quiet 24h window across 5 outlets is a real characteristic of this run, not a bug — see README.md.

Per the brief's own instruction, this trial deliberately does not attempt to scrape 500k records — it proves the pipeline is real end-to-end at a reduced, honest volume, and argues the scaling path below rather than brute-forcing it.

---

1. Scaling to 500k+ records

This trial ran **single-process, single-machine**, against 5 source categories, producing on the order of hundreds to low thousands of records. Scaling is a **horizontal-worker problem, not a code problem**, because every scraper module (`src/scrapers/*.py`) is already an independent async task with no shared mutable state beyond the entity-resolution log and the output writer — both of which already have an obvious distributed form:

- **Shard by source space.** Each scraper's natural pagination axis becomes a work-partition key: different arXiv categories (`cs.AI`, `cs.LG`, `cs.CL`, `cs.CV`, ...) or `start` offset ranges per worker; different YC batch ranges or Algolia facet slices per worker; different job-board identities per worker; different news-feed identities per worker.
- **Run M worker processes** (Celery/RQ, or even a plain `multiprocessing.Pool` / container-per-shard for a trial-adjacent version of "production"), each assigned one shard, all writing through the same dedup layer (§3) so no record is double-counted across workers.
- **Throughput scales linearly with worker count until the *source's* own rate limit becomes the bottleneck** — at that point adding workers stops helping, because the ceiling is the API's, not ours. Concretely:
- **GitHub** (the tightest real constraint we hit): the *search* endpoint used for paper→repo matching is capped at ~30 req/min *per token*, separate from and far tighter than the core API's 5,000 req/hr. The fix that scales here is **provisioning N authenticated tokens and round-robining workers across them** — N tokens gives N × 30 req/min on search and N × 5,000/hr on the core API, linearly.
- **arXiv** self-polices to ~1 req/3s community norm with no hard published cap; horizontal scaling here means more *categories* in flight concurrently (each respecting its own 3s cadence), not more requests per second against one category.
- **Algolia** (YC) has no published hard limit; this run self-imposed ~2 req/sec as a good-citizen default, which is easily parallelized per-facet-shard without hitting a wall in practice.

- **News/Jobs are the easiest to scale**: adding more RSS feeds or more job board integrations is purely additive — each new source is one more independent, fixed-cadence worker, with no cross-source coordination needed beyond the shared dedup store.

---

2. 413 / 429 handling — actual tested behavior

`src/llm/orchestrator.py` implements the 3-tier fallback chain (Gemini → Groq → DeepSeek) with two distinct, deliberately different responses to the two failure modes, both proven live via `python -m src.llm.orchestrator`'s offline simulation (see that module's `__main__` block — it monkeypatches each provider call and asserts the exact fallthrough order):

- **429 (rate limited):** retried on the **same provider**, exponential backoff + jitter via `tenacity.wait_random_exponential(multiplier=1, max=16)`, capped at **4 attempts** (`stop_after_attempt(4)`) — spreading retries across roughly a 1–16 second window before giving up on that provider and falling through to the next one in the chain.
- **413 (payload too large): no same-provider retry** — falls through to the next provider immediately, since retrying an oversized payload against the same provider can't succeed. The chunker (`src/llm/chunker.py`) is applied *before* the first attempt, at a configurable `max_tokens` budget (default 3000), using a paragraph-density scorer rather than naive head-truncation: it keeps lead paragraphs, paragraphs with high numeric density (stats/dates), and paragraphs with apparent named entities, while dropping boilerplate (`length < 20` chars, e.g. "Share"/"Subscribe") and heavily penalizing very long blocks (`length > 2000` chars, likely legal/cookie text). Tested live in this run: a synthetic 4,462-token article truncated to a 60-token budget correctly retained the lead sentence and the highest-density statistic paragraph while dropping filler and boilerplate.
- **GitHub search-endpoint throttling** (a real 429-adjacent constraint discovered during this run, distinct from the LLM orchestrator): initial live testing surfaced that GitHub's `/search/repositories` endpoint has its own ~30 req/min ceiling separate from the core API's 5,000/hr, which the pre-built module hadn't accounted for. Fixed by adding an explicit **2.2s delay between paper-enrichment iterations** (`src/scrapers/arxiv_papers.py`), sized to stay under that endpoint-specific limit — at 300 papers this adds ~11 minutes to a full run, which is an acceptable trial-scale cost for correctness.

---

3. Dedup across distributed runs

For a single-process trial run, in-memory dict-keying by natural id (Algolia `objectID`, arXiv `paper_url`, article URL, job listing URL) is sufficient and is what this run uses. At distributed scale, the same principle generalizes to a **shared, cross-worker dedup store**:

- Compute a **content hash** per record — SHA-256 of the canonical `source.url`, or of `source.url + title` where a source's URLs aren't fully stable (some job boards reuse slugs) — before emitting.
- Check that hash against a **shared set** (Redis `SADD`/`SISMEMBER` for low-latency membership checks at scale, or a `processed_urls` table with a unique constraint if a relational store is already in play) *before* a worker commits a record.
- This is what prevents the same article/paper/listing being processed twice when two workers' shards happen to overlap (e.g. a news feed that briefly appears in two category shards, or a job re-posted across boards with the same canonical URL).
- Entity-level dedup (the *company*, not the *record*) is handled separately and is already implemented and tested in this trial: `EntityCanonicalizer` (`src/resolution/canonicalizer.py`) exact-matches against a seed list first, then fuzzy-matches via `rapidfuzz` (threshold 82.0, tuned against the real "OpenAI" vs "OpenAI" typo case scoring 83.3), logging every decision's method and confidence to the Entity Mapping Log output tab so a human can audit fuzzy merges after the fact.

---

4. Storage justification

This trial: flat JSONL + CSV per entity type (`src/output/writer.py`), chosen deliberately over SQLite/Postgres because the trial's volume (hundreds to low thousands of records, one run, no concurrent writers) doesn't need a database's concurrency guarantees, and JSONL keeps every record trivially diffable, greppable, and re-import-able without any schema migration ceremony. Every record is validated against its pydantic schema before being written; anything that fails validation is rejected and logged to a `*_rejected.jsonl` file rather than silently dropped, so nothing disappears unaccounted for.

At 500k+ scale, this stops being sufficient for two independent reasons — concurrent-write safety across distributed workers, and the actual shape of the "Intelligence Graph" relationships the brief's own name implies:

- **Postgres** as the relational core: gives ACID writes for concurrent workers, a real unique-constraint-backed dedup table (§3), and **JSONB** columns for the fields that genuinely vary in shape by entity type (e.g. a startup's `tags`/`industries` array, a job's raw source payload) without forcing a rigid wide-table schema for data that's naturally semi-structured.
- **pgvector** (staying inside Postgres) if semantic search over paper abstracts / article text becomes a requirement — avoids standing up a separate vector store for a feature that's naturally an extension of the relational store already in place.
- **A dedicated graph DB (Neo4j)** is the stronger argument if the "Intelligence Graph" framing is taken literally: the real value in this dataset is the *relationships* — founders who wrote papers, papers cited by other founders' companies, employees who moved between the startups in the Entity Mapping Log. Queries like "papers by authors who founded YC companies" are multi-hop graph traversals that a relational join chain handles increasingly poorly as hop count grows, while a graph DB makes them a first-class, indexed query pattern. For this trial's flat entity-type outputs (no yet-built cross-entity relationship extraction), Postgres+JSONB is the pragmatic default; Neo4j becomes the right call once relationship extraction between entity types is actually built out.

---

5. Known judgment calls (carried from README.md)

- `PricingModel` has 5 values, not the brief's 4 (`UNKNOWN` added) — YC's directory never exposes pricing; forcing a guess into one of 4 real values would be exactly the hallucination the brief disqualifies for.
- Fuzzy match threshold is 82.0, tuned against a real typo case, not a round number — see §3 above.
- Papers with Code was considered as an additional source alongside Arxiv; Arxiv+GitHub was used since it produces equivalent schema output for this pipeline's purposes.
- The GitHub paper→repo matcher went through three rounds of live-testing fixes, each surfaced by manually sense-checking matched/unmatched samples rather than trusting the first "it ran without erroring" result: 1. The original `sort=stars,order=desc` search matched the same "awesome-list"/daily-paper-tracker aggregator repo to 3 different unrelated papers in one run — fixed by rejecting repo names/descriptions matching known aggregator patterns, rejecting repos with no primary programming language (aggregators are markdown-only), and requiring the paper title to appear verbatim in the actual fetched README content, not just search relevance. 2. That fix's `sort=stars` ordering turned out to actively bury a paper's own brand-new (0-star) repo below older, higher-star, *wrong* aggregator repos — switching to default relevance ordering (dropping `sort=stars`) recovered genuine matches that stricter filtering alone had been missing. 3. Relevance ordering then surfaced a third false-positive class: spam GitHub "profile README" repos (repo name == owner name) padded with dozens of unrelated paper titles for search visibility, plus at least one literally auto-generated tracker repo whose README opened with "AUTO-GENERATED BY PAPERFLOW. DO NOT EDIT MANUALLY." Fixed with an owner==reponame rejection and a content-level check for auto-generation markers in the README's opening 500 characters — deliberately content-based rather than name-based, since it generalizes to aggregators regardless of what they're called. Final state: 113/300 papers (37.7%) matched to a repo, 187 left `null`. This trades recall for precision deliberately — TRD §2.2 is explicit that a wrong repo mapped to a paper is worse than `null` — and the three rounds above are the concrete evidence that precision was actually verified, not assumed.